21CSC574J - BIG DATA ANALYTICS USING ARTIFICIAL INTELLIGENCE TECHNOLOGIES

The file is meant for personal use by dominic897@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

BIG DATA HADOOP CHEAT SHEET

Big Data

Comprises of large datasets that cannot be processed using traditional computing techniques, which includes huge volumes, high velocity and extensible variety of data.

Hadoop

An Apache open source framework written in JAVA which allows distributed processing of large datasets across clusters of computers using simple programming models.

Hadoop Common

These are the JAVA libraries and utilities required by other Hadoop modules which contains the necessary scripts and files required to start Hadoop

Hadoop YARN

A framework used for job scheduling and managing the cluster resources

Hadoop Distributed File System

A Java based file system that provides scalable and reliable data storage and it provides high throughput access to the application data

Hadoop MapReduce

A software framework, which is used for writing the applications easily which process big amount of data in parallel on large clusters

Apache hive

An infrastructure for data warehousing for Hadoop

Apache oozie

An application in Java responsible for scheduling Hadoop jobs

Apache Pig

A data flow platform that is responsible for the execution of the MapReduce jobs

Apache Spark

An open source framework used for cluster computing

Flume

An open source aggregation service responsible for collection and transport of data from source to destination

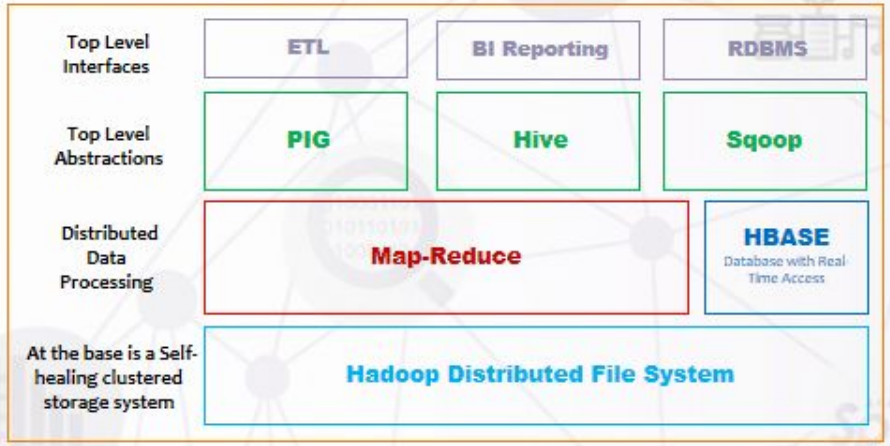
Hbase

A column-oriented database of Hadoop that stores big data in a scalable way

Sqoop

An interface application that is used to transfer data between Hadoop and relational database through commands

Different components of Hadoop Architecture involves:



Hadoop File Automation Commands

Commands	Task	Syntax
cat	Used to copy the source path to the destination or the standard output	hdfsdfs –cat URI [URI- – -]
chgrp	Used to change the group of the files	hdfsdfs –chgrp [-R] GROUP URI [URI—]
chmod	Used to change the permissions of the file	hdfsdfs –chmod [-R] <MODE[,MODE]- – -: OCTALMODE> URI [URI – – -]
chown	Used to change the owner of the file	hdfsdfs –chown [-R][OWNER][:[GROUP]]URI[URI]
count	Used to count the number of directories	hdfs dfs –count [-q] <paths>
cp	Used to copy one or more than one files from the source to destination path	hdfsdfs –cp URI[URI – – -]<dest>
Du	Used to display the size of directories or files	hdfsdfs –du [-s][-h]URI [URI – – -]
get	Used to copy files to the local file system	hdfs dfs –get[-ignorecrc][-crc]<src><localdst>
ls	Used to display the statistics of any file or directory	hdfsdfs –ls <args>
mkdir	Used to create one or more directories	hdfsdfs –mkdir<path>
mv	Used to move one or more files from one location to other	hdfs dfs –mv URI[URI – – -]<dest>
put	Used to read from one file system to other	hdfsdfs –put<localsrc>- – -<dest>
rm	Used to delete one or more than one files	hdfsdfs –rmr[-skipTrash]URI[URI- – –]
stat	Used to display the information of any specific path	hdfsdfs –stat URI[URI – – -]
help	Used to display the usage information of the command	help<cmd-name>

Hadoop Administration Commands

Commands	Task
Balancer	To run cluster balancing utility
daemonlog	To get or set the log level of each daemon
dfsadmin	To run many HDFS administrative operations
Datanode	To run HDFS datanode service
mradmin	To run a number of mapReduce administrative operations
Jobtracker	To run mapReduce job tracker
Namenode	To run name node
Tasktracker	To run mapReduce task tracker node
Secondary namenode	To run secondary namenode

FURTHERMORE:

Big Data Hadoop Certification Training

- Learn from industry experts and be sought-after by the industry!
- Learn any technology, show exemplary skills and have an unmatched career!
- The most trending technology courses to help you fast-track your career!
- Logical modules for both beginners and mid-level learners
- All recorded sessions available in LMS for lifetime
- 24*7 Support for Lifetime
- Learn Anytime, Anywhere

IntelliPaat

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

COURSE LEARNING RATIONALE (CLR)

CLR-1 :	Understand the basics of Hadoop and related technologies to big data analytics and Examine the ecosystem tools in hadoop
CLR-2 :	Provide with comprehensive Apache Spark & Spark SQL knowledge for Big data processing
CLR-3 :	Provide with comprehensive in Spark MLlib and structured streaming
CLR-4 :	Recognize the applications with Apache airflow and Zookeeper
CLR-5 :	Explore the advantages of using Ontologies for enhanced data integration and knowledge representation in Big Data contexts.

This file is meant for personal use by dominic897@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

COURSE LEARNING OUTCOMES (CLO):

AT THE END OF THIS COURSE, LEARNERS WILL BE ABLE TO:

CO-1:	Apply MapReduce, HDFS and YARN develop big data applications
CO-2:	Provide with comprehensive Apache Spark & Spark SQL knowledge for Big data processing
CO-3:	Identify the importance of Spark MLlib and structured streaming
CO-4:	Identify the importance of Apache Airflow and Zookeeper tool
CO-5:	Utilize Ontologies to enhance data integration, interoperability, and knowledge representation in Big Data environments.

The file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Syllabus

Unit 3

Spark MLlib – pySpark – Spark Structured Streaming –Basic concepts, Programming Model, APIs using Streaming Dataframes, Creating Streaming Dataframes, Operations using Streaming dataframes

Lab 7 : Features Engineering using Spark MlLib

Lab 8 : Classification and Regression model building using Spark

Lab 9 : Stream processing using Kafka

The file is meant for personal use by dominic2007@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

1. Spark Machine learning - MLib

Spark MLlib

The Machine Learning Library

MLlib is a machine learning framework built on top of Apache Spark.

It is designed to make machine learning tasks faster and more efficient.

MLlib supports various ML algorithms and utilities.

The file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

1. Spark Machine le

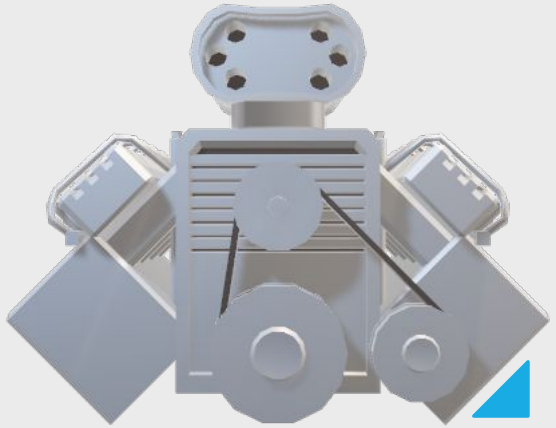
MLib

ib is Spark's **Machine Learning Library** inside:

Apache Spark

It provides algorithms like:

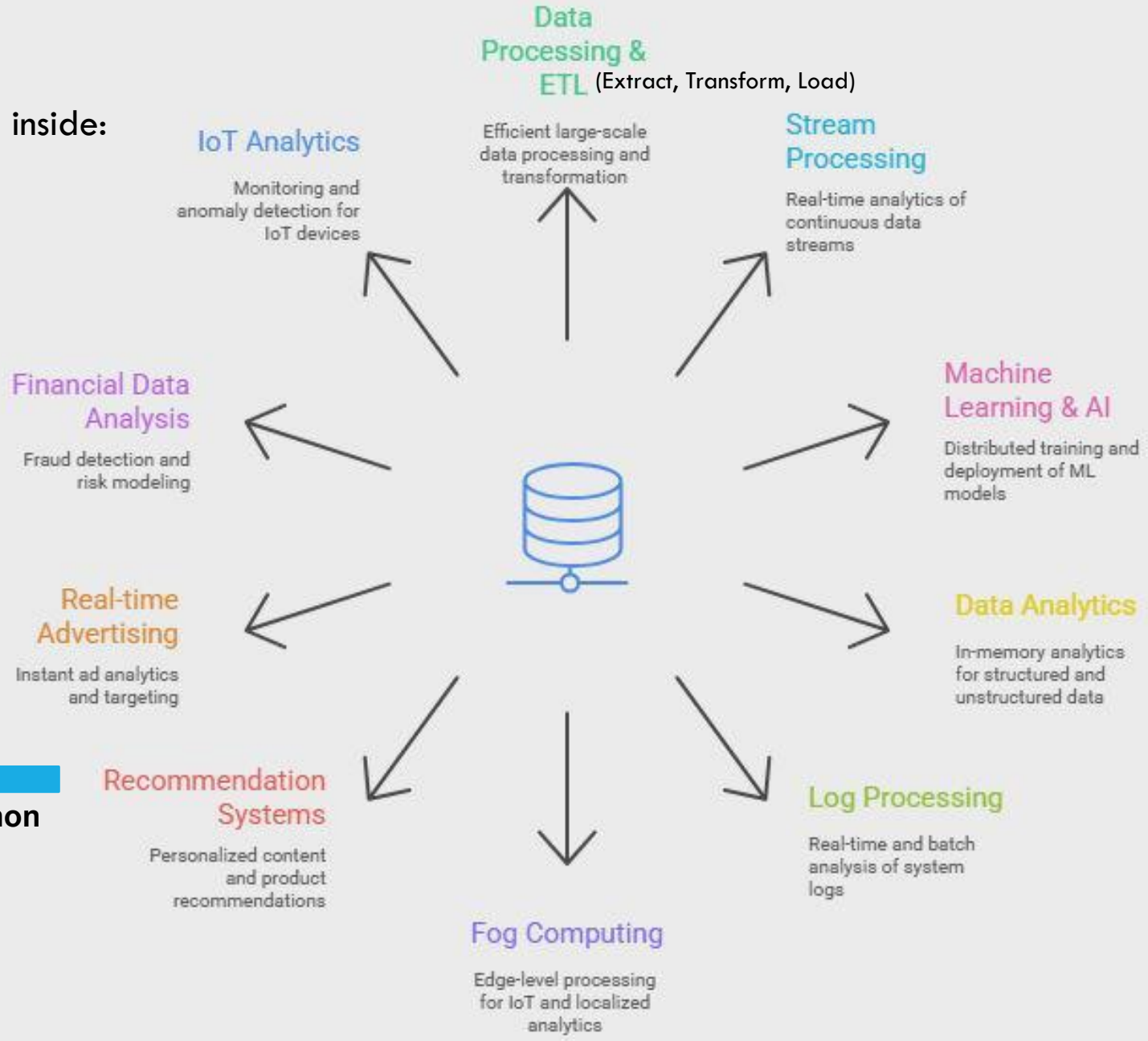
- Logistic Regression
- Random Forest
- K-Means
- Decision Trees
- Recommendation systems



Spark = Engine

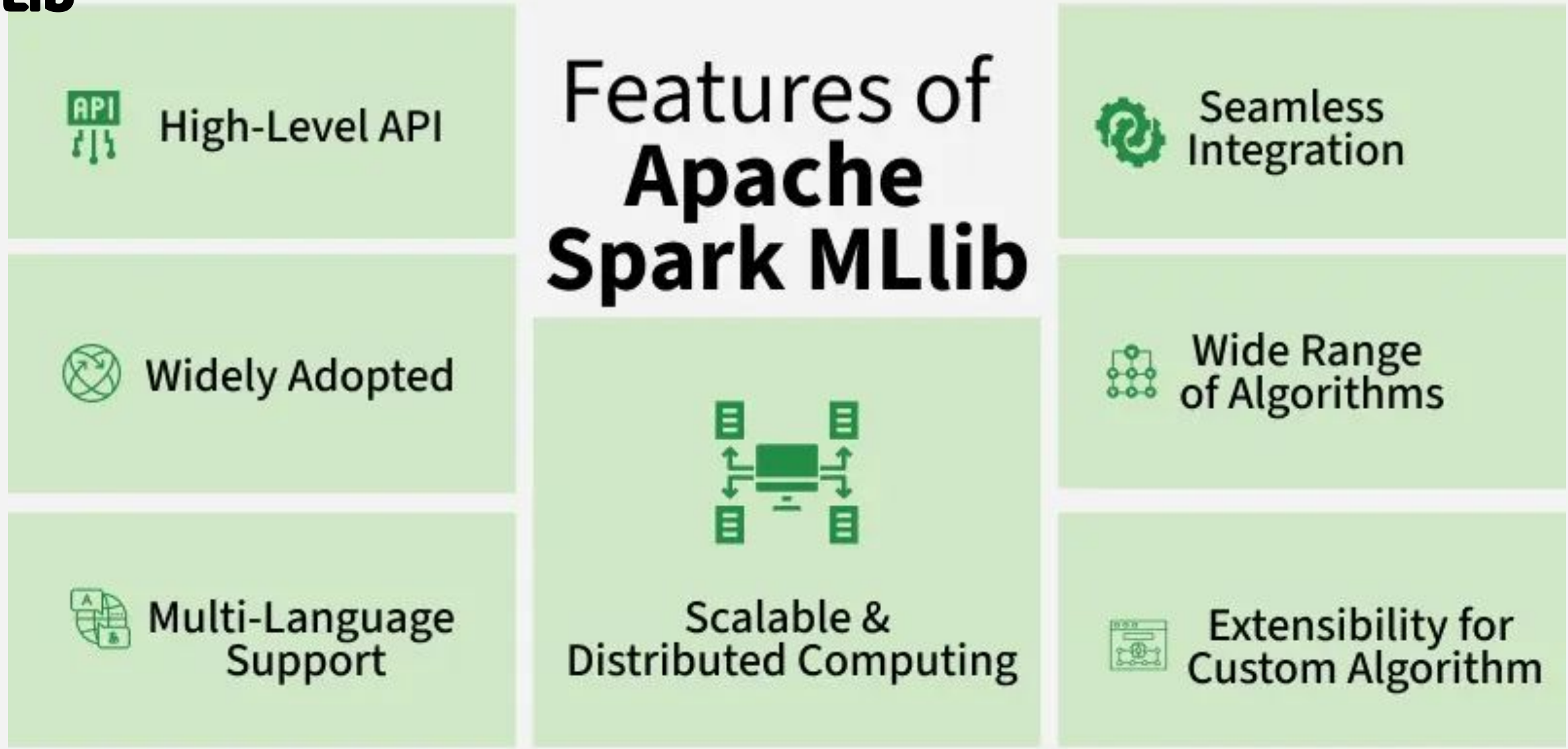
MLlib = Machine learning tools inside engine

Scala / Python = Steering wheel



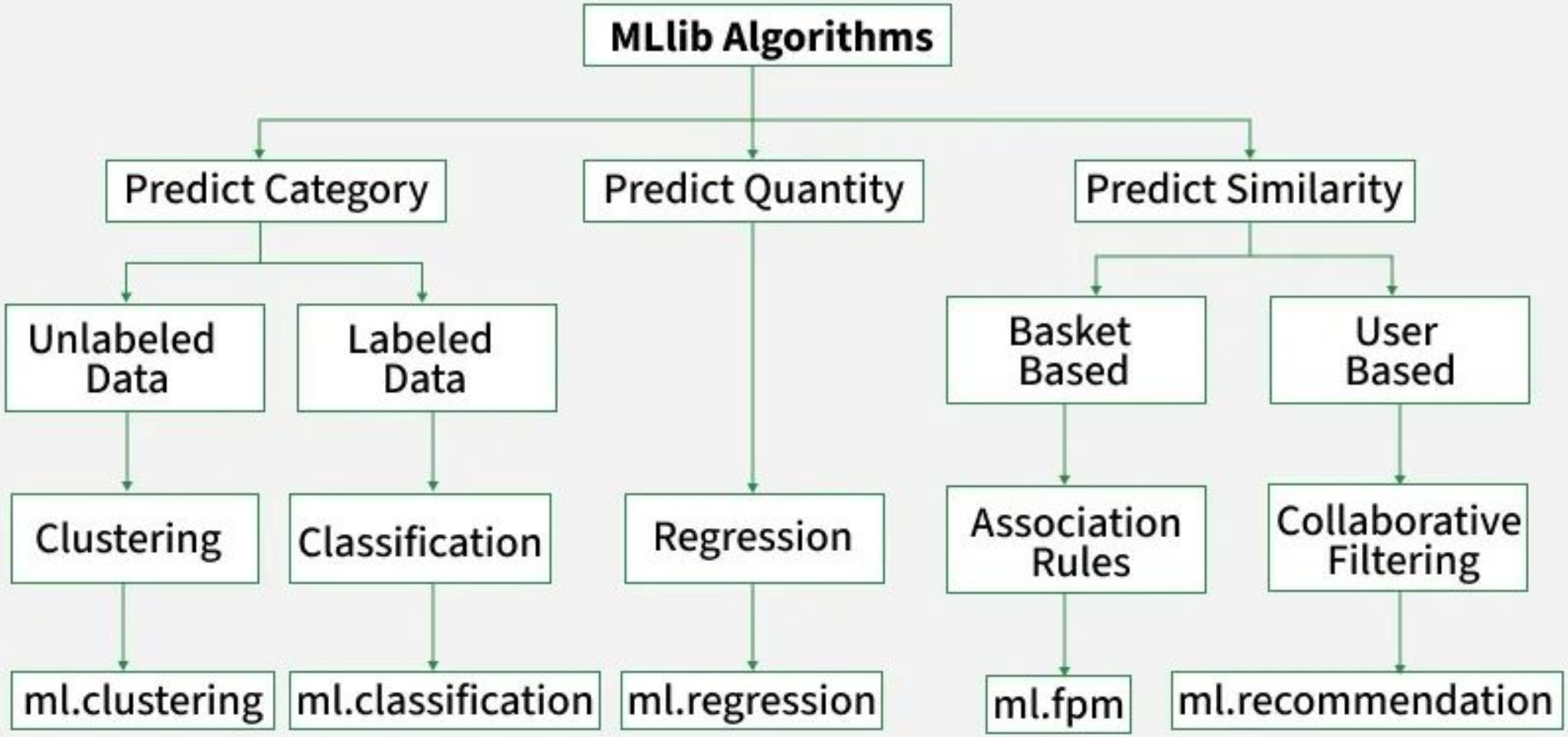
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

1. Spark Machine learning - MLib



The file is meant for personal use by dominic007@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

1. Spark Machine learning - MLib



The file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

1. Spark Machine learning - MLib

Workflow of MLlib Models

- **Data Ingestion**: Load data using Spark DataFrames.
- **Data Preprocessing**: Cleaning, handling missing values. Feature selection and transformation
- **Model Selection**: Choose a machine learning algorithm based on data.
- **Training**: Fit the model on training data.
- **Prediction**: Use the trained model to make predictions on test or new data.
- **Evaluation**: Evaluate model performance using various metrics.
- **Pipeline Deployment**: Build a pipeline combining all steps.

The file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

1. Spark Machine learning - MLib

Spark MLlib Tools

ML Algorithms: ML Algorithms form the core of MLlib.

These include common learning algorithms such as classification, regression, clustering and collaborative filtering.

Featurization: Featurization includes feature extraction, transformation, dimensionality reduction and selection.

Pipelines: Pipelines provide tools for constructing, evaluating and tuning ML Pipelines.

Persistence: Persistence helps in saving and loading algorithms, models and Pipelines.

Utilities: Utilities for linear algebra, statistics and data handling.

The file is meant for personal use by dominic2007@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Aspect	PySpark	Spark MLlib (MLlib)
What it is	Python API for **entire Spark**	Spark's **machine learning library**
Scope	Whole Spark ecosystem	Only the machine learning part
Contains	Spark SQL, DataFrame, Spark Streaming, Spark MLlib, GraphX, etc.	Classification, Regression, Clustering, Recommendation, Feature engineering, Pipeline API, etc.
Package	`pyspark`	`pyspark.ml` + `pyspark.mllib`
Main import paths	`from pyspark.sql import SparkSession from pyspark.sql.functions import col`	`from pyspark.ml.classification import RandomForestClassifier from pyspark.mllib.clustering import KMeans`
Purpose	General purpose data processing in Python	Specifically doing machine learning at scala
Can you use one without the other?	Yes (you can use PySpark without ML)	No — you almost always use it **through PySpark**

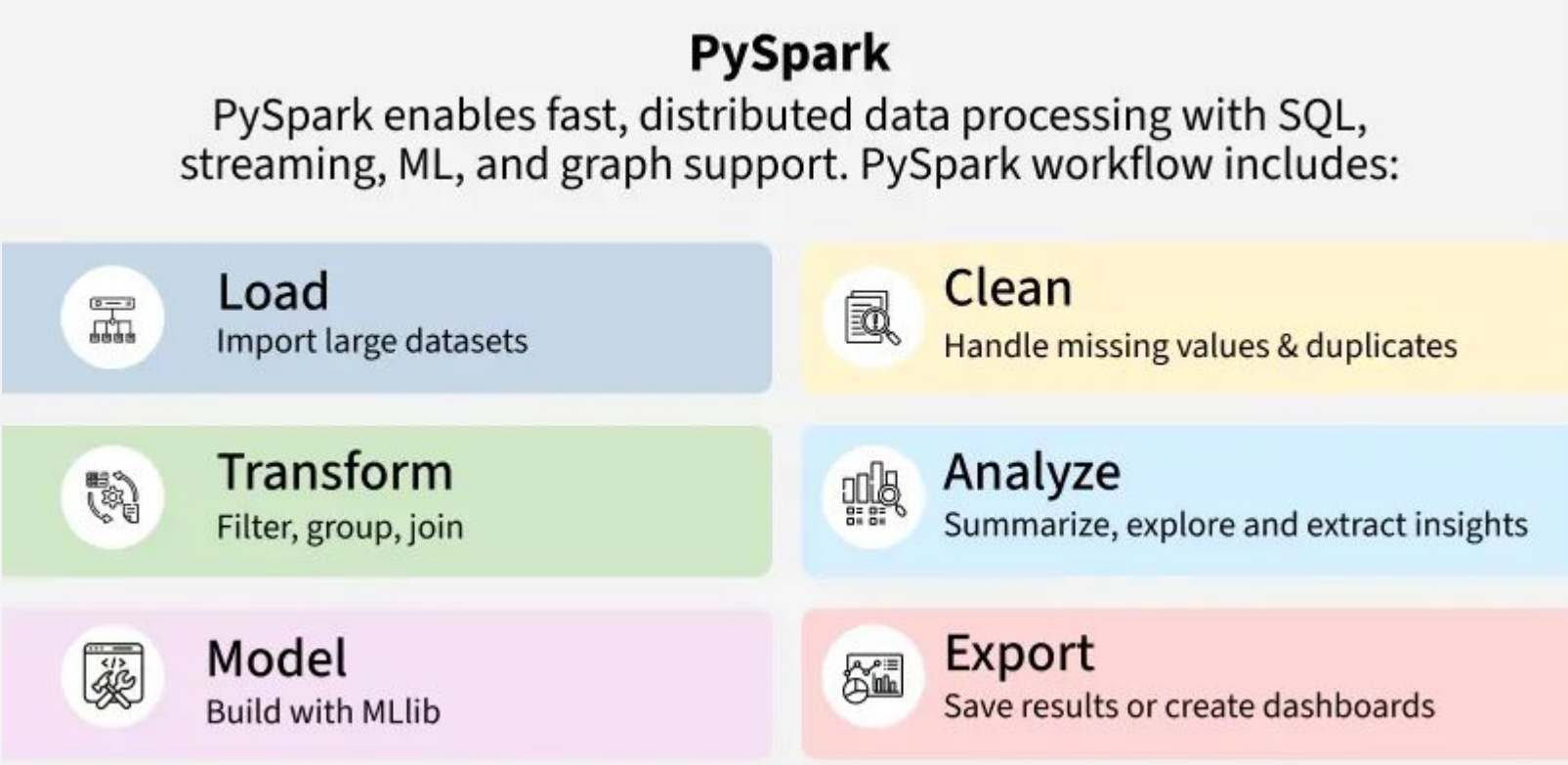
The file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

#	Aspect	Scala (spark-shell)	PySpark (pyspark)
1	Object Creation	Uses `new` keyword	No `new` keyword
	Code Example	`val lr = new LogisticRegression()`	`lr = LogisticRegression()`
2	Setting Parameters	Uses `.set` methods	Uses arguments inside constructor
	Code Example	`.setFeaturesCol("features")`	`featuresCol="features"`
3	Array Declaration	Uses `Array()`	Uses Python list `[]`
	Code Example	`setInputCols(Array("a","b"))`	`inputCols=["a","b"]`
4	DataFrame Creation	Uses `Seq().toDF()`	Uses Python list directly
	Code Example	`Seq((5.0,80.0,1.0)).toDF(...)`	`[(5.0,80.0,1.0)]`
5	Typing System	Strongly typed (val, var)	Dynamically typed (no type declaration)
	Code Example	`val df = ...`	`df = ...`

The file is meant for personal use by dominic2097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

2. pySpark

- PySpark is the Python API for Apache Spark, designed for big data processing and analytics.
- It lets Python developers use Spark's powerful distributed computing to efficiently process large datasets across clusters.
- It is widely used in data analysis, machine learning and real-time processing.



The file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

2. pySpark

PySpark Core Modules

PySpark provides core modules for large-scale data processing, machine learning, and real-time streaming. Key modules include:

SParkContext (SC)
Main Entry Point

SparkSession (entry point)
Unified Entry Point

RDDs
Resilient Distributed Datasets

DataFrames
Distributed Table

SQL
Querying Data

MLlib
Machine Learning library

Spark Streaming
Real-Time Processing

The file is meant for personal use by dominic0097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

2. pySpark

- **Python Spark Connect Client**
  - Spark Connect is a client-server architecture within Apache Spark that enables remote connectivity to Spark clusters from any application. PySpark provides the client for the Spark Connect server, allowing Spark to be used as a service.
- **Spark SQL and DataFrames**
  - Spark SQL is Apache Spark's module for working with structured data.
  - It allows you to seamlessly mix SQL queries with Spark programs.
  - With PySpark DataFrames you can efficiently read, write, transform, and analyze data using Python and SQL.
  - Whether you use Python or SQL, the same underlying execution engine is used so you will always leverage the full power of Spark.

The file is meant for personal use by dominic897@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

2. pySpark

- **Pandas API on Spark**
  - Pandas API on Spark allows you to scale your panda's workload to any size by running it distributed across multiple nodes.
  - If you are already familiar with pandas and want to leverage Spark for big data, pandas API on Spark makes you immediately productive and lets you migrate your applications without modifying the code.
  - You can have a single codebase that works both with pandas (tests, smaller datasets) and with Spark (production, distributed datasets) and you can switch between the pandas API and the Pandas API on Spark easily and without overhead.
  - Pandas API on Spark aims to make the transition from pandas to Spark easy but if you are new to Spark or deciding which API to use, we recommend using PySpark

The file is meant for personal use by dominic897@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

2. pySpark

- **Structured Streaming**
  - Structured Streaming is a scalable and fault-tolerant stream processing engine built on the Spark SQL engine.
  - You can express your streaming computation the same way you would express a batch computation on static data.
  - The Spark SQL engine will take care of running it incrementally and continuously and updating the final result as streaming data continues to arrive.
- **Machine Learning (MLlib)**
  - Built on top of Spark, MLlib is a scalable machine learning library that provides a uniform set of high-level APIs that help users create and tune practical machine learning pipelines.
- **Declarative Pipelines**
  - Spark Declarative Pipelines (SDP) is a declarative framework for building reliable, maintainable, and testable data pipelines on Spark.
  - SDP simplifies ETL development by allowing you to focus on the transformations you want to apply to your data, rather than the mechanics of pipeline execution.

The file is meant for personal use by dominic897@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

2. pySpark

- **Spark Core and RDDs**
  - Spark Core is the underlying general execution engine for the Spark platform that all other functionality is built on top of. It provides RDDs (Resilient Distributed Datasets) and in-memory computing capabilities.
  - Note that the RDD API is a low-level API which can be difficult to use and you do not get the benefit of Spark's automatic query optimization capabilities.
  - We recommend using DataFrames instead of RDDs as it allows you to express what you want more easily and lets Spark automatically construct the most efficient query for you.
- **Spark Streaming (Legacy)**
  - Spark Streaming is an extension of the core Spark API that enables scalable, high-throughput, fault-tolerant stream processing of live data streams.
  - Note that Spark Streaming is the previous generation of Spark's streaming engine.
  - It is a legacy project and it is no longer being updated.
  - There is a newer and easier to use streaming engine in Spark called Structured Streaming which you should use for your streaming applications and pipelines.

The file is meant for personal use by dominic897@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

2. pySpark

Statistics & Key Concepts

Speed & Performance

Up to 100x faster with in-memory computing.

Scalability

Runs on a single machine or large clusters.

Pythonic & Easy

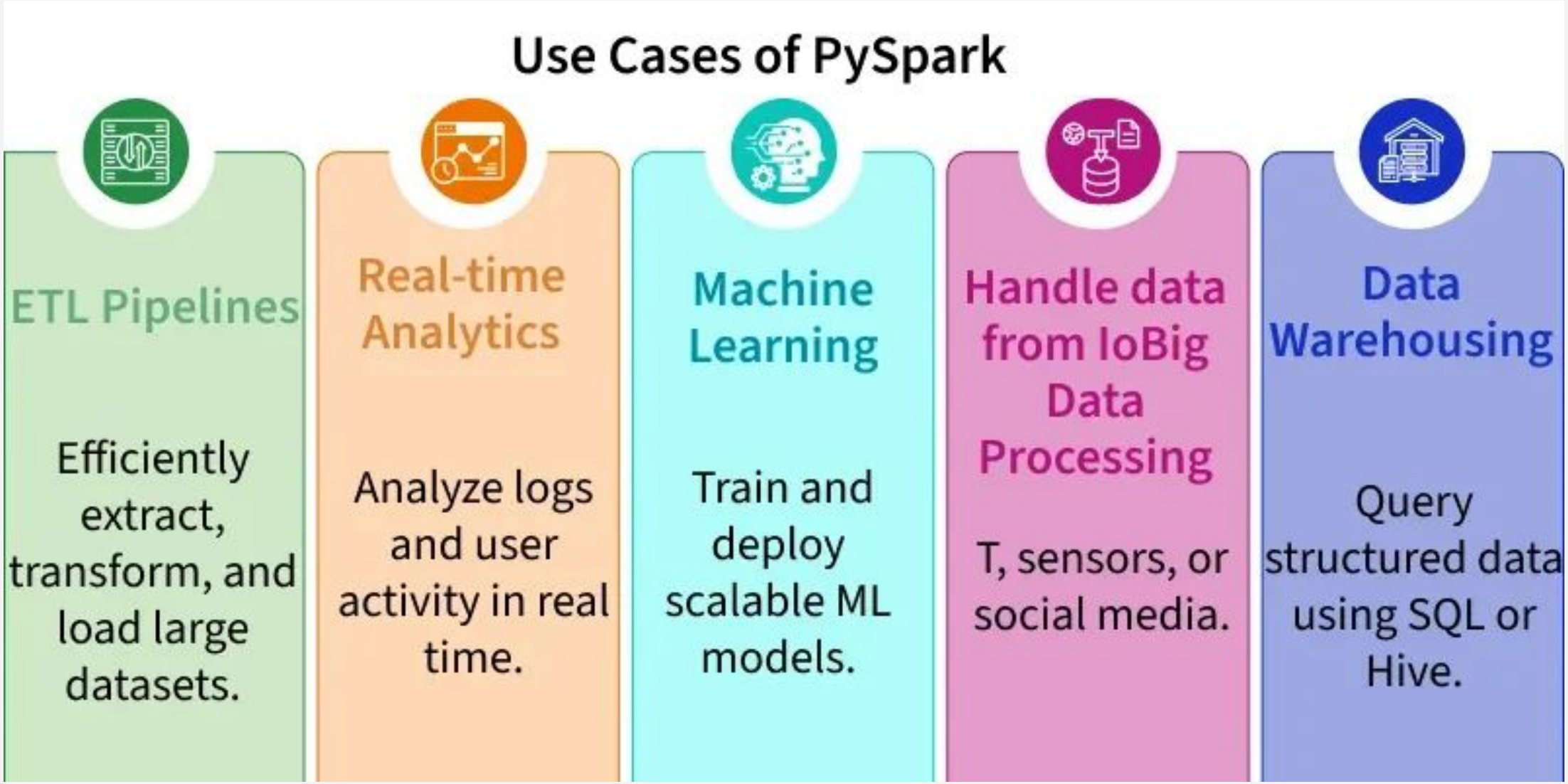
Use familiar Python functions like filter(), groupBy().

ML & Streaming

Built-in support for MLlib and real-time streaming.

The file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

2. pySpark



The file is meant for personal use by dominic007@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

2. pySpark

Features of PySpark

The file is meant for personal use by dominic2097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

3. Spark Structured Streaming

- Structured Streaming is a scalable, fault-tolerant stream processing engine built on top of the SQL engine of Apache Spark.
- **It enables real-time processing of continuous streams of data using the same APIs used for batch processing with DataFrames and SQL**.
- Unlike traditional batch systems where datasets are static, structured streaming processes unbounded datasets that continuously grow over time.

Processes real-time data streams at scale.

Ensures fault tolerance and high availability.

Supports exactly-once processing semantics.

Enables real-time analytics, monitoring, and event-driven architectures.

The file is meant for personal use by dominic897@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

3. Spark Structured Streaming

SRM INSTITUTE OF SCIENCE & TECHNOLOGY
Deemed to be University u/s 3 of UGC Act, 1956

Need for Structured Streaming

- Traditional data processing methods analyze **static datasets.**
- However, modern applications require processing **continuously generated data.**

Example

Application	Streaming Data
Stock market	Live stock price updates
Social media	Continuous tweets and posts
IoT devices	Sensor readings
Online shopping	Real-time purchase events
Traffic systems	GPS data from vehicles

The file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

3. Spark Structured Streaming

Key Features of Spark Structured Streaming

1. Real-Time Data Processing

Structured Streaming processes data continuously in **near real-time.**

Example input stream:

- AI
- Spark
- Machine Learning
- AI

Output:

- AI → 2
- Spark → 1
- Machine Learning → 1

2. Unified Batch and Streaming Processing

Structured Streaming uses the **same APIs for batch and streaming data.**

Example:

Batch Processing	Streaming Processing
spark.read()	spark.readStream()
spark.write()	spark.writeStream()

The file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

3. Spark Structured Streaming

Key Features of Spark Structured Streaming

3. Micro-Batch Processing

Spark processes streaming data in **small batches called micro-batches.**

Example:

Spark processes these records in small time intervals.

4. Scalability

Structured Streaming can run across **multiple machines in a cluster,** allowing it to process **large volumes of streaming data** efficiently.

5. Fault Tolerance

Spark ensures reliability through **checkpointing and write-ahead logs,** which help recover data in case of failures.

The file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

3. Spark Structured Streaming

Architecture of Structured Streaming

The general architecture consists of four main stages:

Example:

Streaming Source → Streaming DataFrame → Data Transformations → Output Sink

- **Streaming Source**
  The system from which streaming data is received.
  - Examples:
    - Apache Kafka
    - Socket streams
    - File directories
    - IoT sensors
- **Streaming DataFrame**
  - The incoming data is represented as a **Streaming DataFrame**, which behaves like a normal Spark DataFrame but continuously receives new data.
- **Transformations**
  - Operations such as filtering, grouping, and aggregation are applied to the streaming data.
- **Output Sink**
  - The processed results are written to destinations such as files, databases, dashboards, or the console.

This file is meant for personal use by dominic0907@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

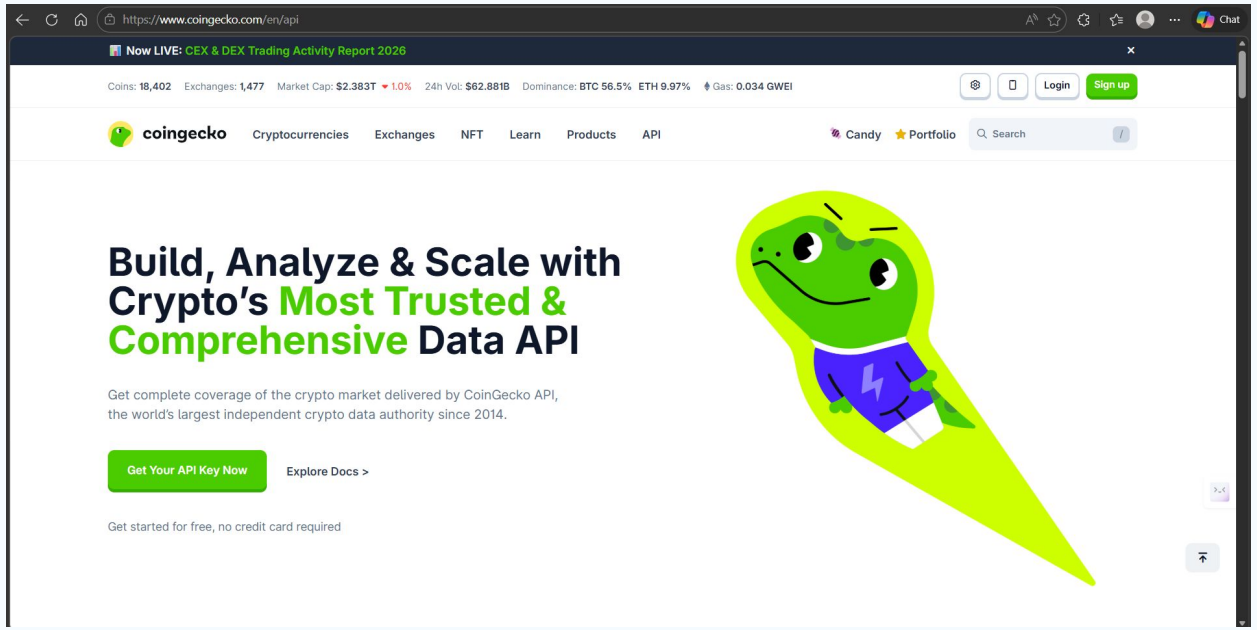
3. Spark Structured Streaming

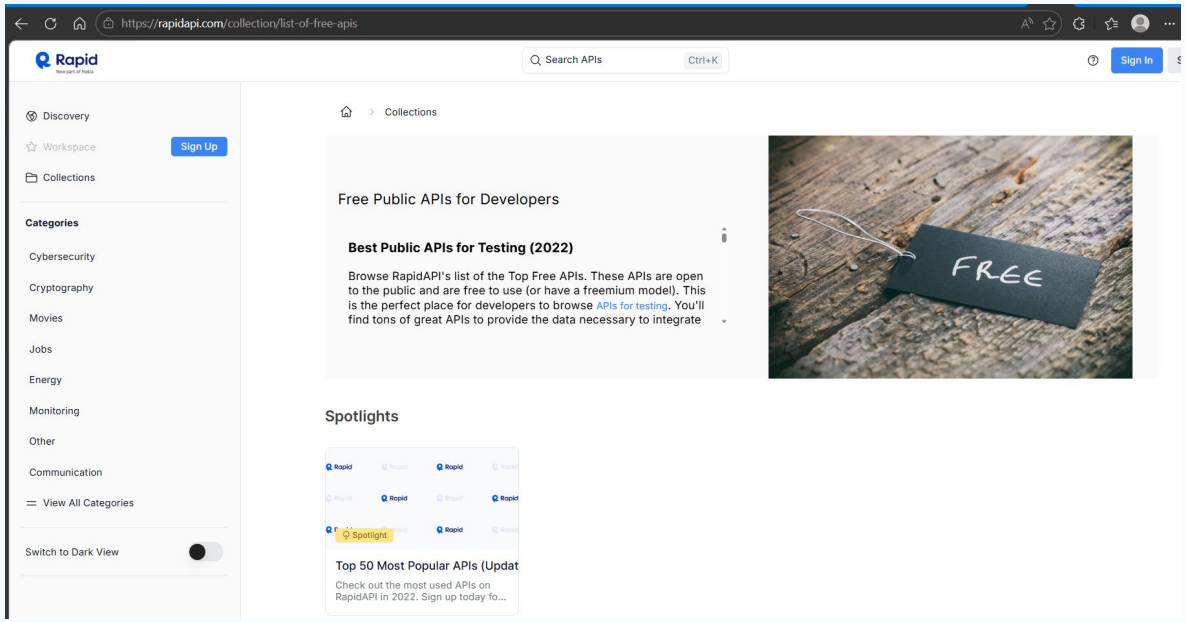
Real-World Applications

Financial Analytics	Detect fraudulent transactions in real time.
IoT Monitoring	Analyze data from sensors and smart devices.
Social Media Analytics	Track trending hashtags and user interactions.
Traffic Monitoring	Analyze real-time traffic flow using GPS data.
E-commerce Systems	Track customer purchases and product recommendations in real time.

The file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

4. APIs using Streaming Dataframes



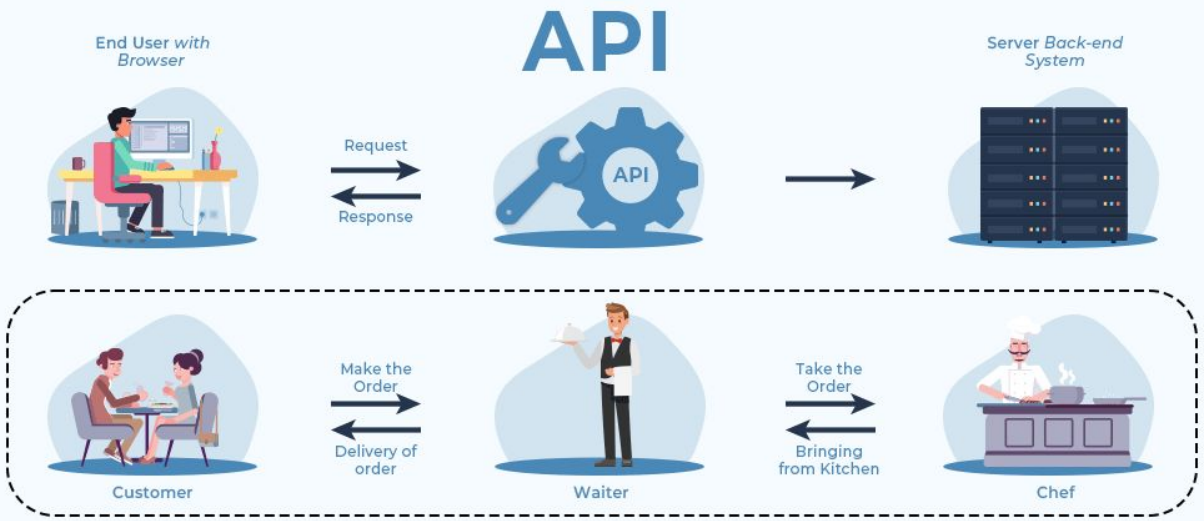


This file is meant for personal use by dominic897@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

4. APIs using Streaming Dataframes

- In **Spark Structured Streaming**, streaming data is processed using the **same DataFrame APIs used for batch processing**.
- These APIs allow developers to perform operations such as filtering, aggregation, grouping, and transformation on continuously arriving data streams.
- A **Streaming DataFrame** represents an **unbounded table**, meaning new rows are continuously added as new data arrives.
- Using these APIs, Spark can process real-time data efficiently with the same programming model used for batch analytics.

- **Streaming DataFrame APIs** are a set of functions provided by Spark Structured Streaming that allow real-time data streams to be processed using **DataFrame and SQL operations.**
- These APIs make it possible to perform streaming analytics using familiar operations like `select`, `filter`, `groupBy`, and `count`.



The file is meant for personal use by dominic897@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

4. APIs using Streaming Dataframes

Key APIs Used in Streaming DataFrames

API	Purpose
readStream()	Reads data from streaming sources
writeStream()	Writes streaming output to a sink
select()	Selects specific columns
filter()	Filters streaming data
groupBy()	Groups streaming records
count()	Counts occurrences
agg()	Performs aggregations
join()	Combines streaming datasets

The file is meant for personal use by dominic2007@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

4. APIs using Streaming Dataframes

Advantages of Using Streaming DataFrame APIs

- Same API for **batch and streaming processing**
- Easy to implement **real-time analytics**
- Supports **SQL queries and machine learning**
- Highly scalable across clusters
- Simplifies complex stream processing

The file is meant for personal use by dominic897@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

4. APIs using Streaming Dataframes

SRM INSTITUTE OF SCIENCE & TECHNOLOGY
Deemed to be University u/s 3 of UGC Act, 1956

Python code

Step 1 — Install Library

pip install requests pandas

Step 2 — Streaming Program

Create file:
nano crypto_stream_demo.py
Code:

```
import requests
import pandas as pd
import time

url = "https://api.coingecko.com/api/v3/coins/markets"

params = {
"vs_currency": "usd",
"order": "market_cap_desc",
"per_page": 10,
"page": 1
}

print("Starting Crypto Streaming...\n")

while True:

response = requests.get(url, params=params)

data = response.json()

df = pd.DataFrame(data)

print("\nLive Crypto Market Data\n")

# SELECT operation
selected = df[['name','current_price','symbol']]

print("SELECT Example (coin name, price, symbol)")
print(selected)

# FILTER operation
high_price = df[df['current_price'] > 1000]

print("\nFILTER Example (price > 1000)")
print(high_price[['name','current_price']])

# GROUP BY operation
grouped = df.groupby('symbol')

print("\nGROUP BY Example (symbol groups)")
print(grouped.size())

# COUNT operation
print("\nCOUNT Example")
print("Total coins fetched:", df['name'].count())

time.sleep(10)
```

Step 3 — Run Program

python3 crypto_stream_demo.py
Every **10 seconds the program fetches new market data**.

Example Output

```
SELECT Example
----------------
name price symbol
Bitcoin 62000 btc
Ethereum 3200 eth
Solana 150 sol
FILTER Example (price > 1000)
-----------------------------
Bitcoin 62000
Ethereum 3200
GROUP BY Example
----------------
btc : 1
eth : 1
sol : 1
COUNT Example
-------------
Total coins fetched : 10
```

Enter code: nano crypto_stream_demo.py

Execute code: python3 crypto_stream_demo.py

Operation	What it does
SELECT	choose specific columns
FILTER	select rows meeting condition
GROUP BY	group similar records
COUNT	count number of records

This file is meant for personal use by dominic897@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

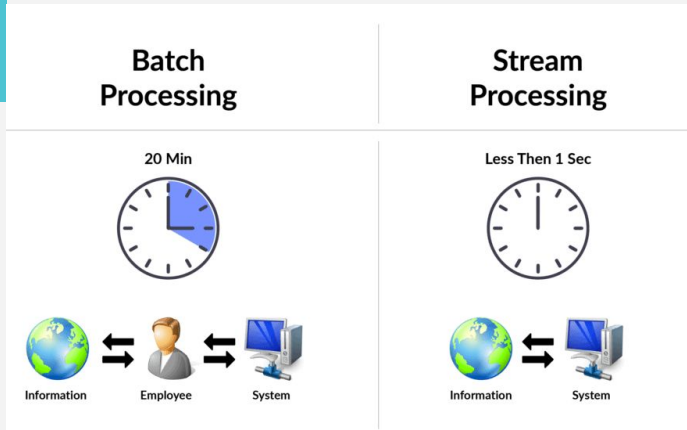
5. Creating Streaming Dataframes

- In **Spark Structured Streaming,** data from a streaming source must first be converted into a **Streaming DataFrame.**
- A Streaming DataFrame represents **continuous incoming data** and allows Spark to process the data in real time using DataFrame operations.
- Unlike a normal DataFrame that works on **static datasets,** a Streaming DataFrame works on **unbounded datasets,** meaning new data continuously arrives.

- A **Streaming DataFrame** is a DataFrame that continuously receives data from a streaming source and allows real-time processing using Spark Structured Streaming.
- It is created using the **`readStream()` API**.

The file is meant for personal use by domino297@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

5. Creating Streaming Dataframes



Batch DataFrame vs Streaming DataFrame

Batch DataFrame

- Fixed, bounded dataset
- Data loaded once from disk
- Uses spark.read()
- Processes all data at once
- Results written once
- No continuous execution

Streaming DataFrame

- Unbounded, continuous data
- Data arrives from live sources
- Uses spark.readStream()
- Processes new data incrementally
- Results written per trigger
- Runs continuously until stopped

The file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

5. Creating Streaming Dataframes

4 Key Characteristics of Streaming DataFrames

1. Unbounded Nature

Streaming DataFrames are designed to handle data that arrives indefinitely with no predefined end. The stream never 'completes' — it grows continuously as new events occur in real-time systems like IoT sensors, web logs, or financial transactions.

2. Unified API

Shares the same API as batch DataFrames. Operations like filter(), groupBy(), select(), join(), and window() work identically in both batch and streaming contexts — enabling massive code reuse and a flat learning curve for developers.

3. Fault Tolerance

Built-in checkpointing saves the streaming query state to reliable storage (HDFS, S3, ADLS). If the application crashes or a node fails, the query automatically resumes from the last saved checkpoint without reprocessing data.

4. Incremental Processing

Only newly arrived data is processed in each micro-batch trigger. Already-processed data is never re-read. This makes streaming queries highly efficient — resource usage scales with incoming data rate, not total historical data size.

The file is meant for personal use by dominic2097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

5. Creating Streaming Dataframes

Apache Spark treats a stream as an unbounded table — new data is continuously appended as new rows.

Data Source	Streaming DF	Your Query	Result Table	Output Sink
Kafka / Files Socket / Rate	Ever-growing unbounded table	filter / groupBy window / join	Updated per trigger	Console / File Kafka / Memory

How Micro-Batching Works

Micro-batch 1
Row 1, Row 2, Row 3

Micro-batch 2
Row 4, Row 5

Micro-batch 3
Row 6, Row 7, Row 8

Each trigger interval = one micro-batch. Only NEW rows are processed per batch. Already-processed data is tracked and skipped.

Key Insight: spark.readStream uses readStream instead of read. df.isStreaming returns True for all Streaming DataFrames.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

5. Creating Streaming Dataframes

SRM INSTITUTE OF SCIENCE & TECHNOLOGY
Deemed to be University u/s 3 of UGC Act, 1956

Creating a Streaming DataFrame — 4 Input Sources

Use spark.readStream to create a Streaming DataFrame. PySpark supports 4 main input sources:

FILE SOURCE — PRODUCTION

- **Monitors a directory for new files (CSV, JSON, Parquet, ORC).**
- Each new file dropped = one micro-batch.
- Best for: log files, ETL pipelines, file-based data feeds.

KAFKA SOURCE — PRODUCTION

- **Reads from Apache Kafka topics in real-time.**
- High-throughput, distributed, fully fault-tolerant.
- Best for: event streaming, clickstreams, IoT telemetry.

SOCKET SOURCE — TESTING ONLY

- **Reads text lines from a TCP network socket.**
- NOT fault-tolerant — do not use in production.
- Best for: quick prototyping and pipeline testing.

RATE SOURCE — BENCHMARKING

- **Auto-generates rows at a configurable rate (rows/sec).**
- No external dependency needed — built into Spark.
- Best for: load testing and performance benchmarking.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

5. Creating Streaming Dataframes

Writing Streaming Output — Sinks & Output Modes

D
Console Sink
Debugging
`.format("console")`

A
File Sink
Archival
`.format("parquet")`

P
Kafka Sink
Pub/Sub
`.format("kafka")`

Q
Memory Sink
Query/Test
`.format("memory")`

Output Modes

APPEND

Outputs ONLY new rows added since the last trigger. Default mode. Cannot be used with aggregations that change existing results.

Use when: Simple transforms, no aggregation

COMPLETE

Outputs the ENTIRE result table on every trigger. Required when aggregations update existing rows (e.g., running count). Higher memory usage.

Use when: groupBy aggregations

UPDATE

Outputs only the rows that CHANGED since the last trigger. More efficient than complete for large result tables where only some rows change.

Use when: Stateful aggregations

The file is meant for personal use by dominic897@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

5. Creating Streaming Dataframes

Performance Optimization & Best Practices

1 Tune Partitioning

Use repartition() to balance data evenly across executor nodes. Unbalanced partitions cause some tasks to run much longer, slowing the entire batch and wasting cluster resources.

2 Optimize Triggers

Choose trigger intervals based on latency requirements. Fixed intervals prevent too-frequent batches (overhead) or too-rare batches (increased latency). Match interval to your SLA.

3 Leverage Caching

Cache intermediate results reused across transformations using cache() or persist(). Avoids recomputing expensive operations on every micro-batch trigger cycle.

4

Handle skew where one partition gets significantly more data. Skew causes straggler tasks. Use salting techniques, repartitionByRange(), or skew hints.

5

Always set checkpointLocation for ALL production queries. Without it, any failure means starting from scratch — potentially losing data or duplicating output records.

6

Apply filter() immediately after readStream — before any joins or aggregations. This minimizes data volume flowing through the most expensive downstream pipeline stages.

The file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

5. Creating Streaming Dataframes

Streaming DataFrame API Cheat Sheet

`spark.readStream`	Start creating a Streaming DataFrame
`.format("csv"|"kafka"...)`	Set the input source type
`.schema(schema)`	Define the data structure (required)
`.option("key", "value")`	Pass source-specific configuration
`.load("/path")`	Finalize and return Streaming DF
`df.isStreaming`	Returns True if DF is streaming
`df.filter(condition)`	Filter rows by condition
`df.select("col1","col2")`	Choose specific columns
`df.groupBy().agg()`	Aggregate grouped data
`window("col","N minutes")`	Group by time window

`df.join(static, "id")`	Join with static DataFrame
`df.writeStream`	Start writing streaming output
`.outputMode("append")`	Set output mode (append/complete/update)
`.format("console")`	Set output sink format
`.trigger(processingTime)`	Set trigger interval
`.option("checkpointLocation")`	Set checkpoint directory path
`.start()`	Start the streaming query
`query.awaitTermination()`	Keep query running indefinitely
`query.status`	Get current query status dict
`query.stop()`	Stop the streaming query

The file is meant for personal use by dominic007@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

5. Creating Streaming Dataframes

Syntax for Creating Streaming DataFrame

```
spark.readStream
    .format(source)
    .option(key,value)
    .load()
```

Component	Description
spark	SparkSession object
readStream()	reads streaming data
format()	defines streaming source
option()	configuration settings
load()	loads streaming data

The file is meant for personal use by dominic0917@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

6. Operations using Streaming dataframes

- Once a **Streaming DataFrame** is created from a streaming source, Spark allows various **operations (transformations and aggregations)** to be performed on the streaming data.
- These operations are similar to the operations used in **batch DataFrames**, but they operate on continuously arriving data.
- Spark Structured Streaming processes these operations **incrementally**, meaning new data is processed as it arrives without reprocessing the entire dataset.

Operations using Streaming DataFrames refer to the transformations and aggregations performed on streaming data using DataFrame APIs such as `select()`, `filter()`, `groupBy()`, `count()`, `join()`, and `agg()`.
These operations allow real-time analytics on continuous data streams.

Types of Operations

Operations in Structured Streaming can be categorized into:

Type	Description
Transformations	Modify streaming data
Aggregations	Perform calculations on grouped data

The file is meant for personal use by dominic0907@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

6. Operations using Streaming dataframes

SRM INSTITUTE OF SCIENCE & TECHNOLOGY
Deemed to be University u/s 3 of UGC Act, 1956

Advantages of Using Operations on Streaming DataFrames

- Enables real-time analytics on continuous data streams.
- Uses the same APIs as batch DataFrames.
- Supports complex transformations and aggregations.
- Scales efficiently for large data streams.

The file is meant for personal use by dominic897@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

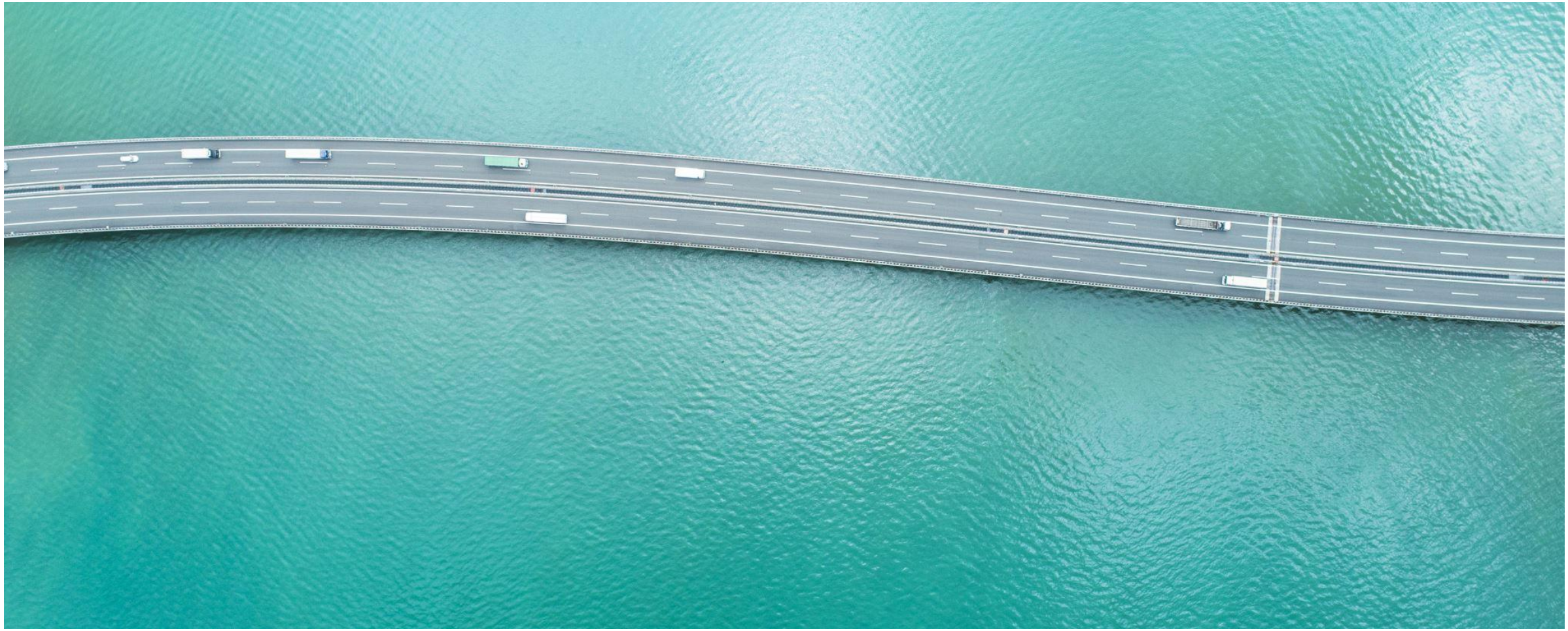
6. Operations using Streaming dataframes

Common Operations on Streaming DataFrames

Operation	Purpose
select()	select specific columns
filter()	filter rows based on condition
groupBy()	group data for aggregation
count()	count records
agg()	perform aggregation
join()	combine streaming datasets

The file is meant for personal use by dominic897@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Thank you!



This file is meant for personal use by dominic007@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.